

Heap

You are given an unsorted array of n integers and an integer k .

Write a program to find the k -th smallest element in the array using a heap-based approach.

Solution (Heap):

Instead of sorting the entire array, use a **max heap of size k** .

Process:

- Insert the first K elements into a max heap.
- Traverse the remaining elements.
- If the current element is smaller than the heap's top (largest of the k smallest so far), remove the top and insert the new element.
- After processing all elements, the heap top is the **k -th smallest element**.

Time Complexity:
 $O(n \log k)$

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

int kSmallest(vector<int> &arr, int k);

int main {
    vector<int> arr = { 7, 10, 4, 3, 20, 15 };
    int k = 3;

    cout << "kth smallest item : " << kSmallest(arr, k) << endl;
    return 0;
}
```

3

```
int kSmallest(vector<int> &arr, int k) {
    priority_queue<int> maxHeap;
```

// Insert first k items

```
for (int i = 0; i < k; ++i) {
    maxHeap.push(arr[i]);
}
```

3

// Traverse and compare

```
for (int i = k; i < arr.size(); ++i) {
    if (arr[i] < maxHeap.top()) {
```

1 way [maxHeap.pop();
maxHeap.push(arr[i]);

way we did [H[0] = arr[i]
bubble down

3

3

return maxHeap.top(); // imagine we have a .top() that returns
root of heap

3

